KIT-PROJECTS · STARTER · ANNOTATED SKETCH

# Modulino starter sketch

The smallest useful Modulino program: press button A to light a pixel green, press B to turn it off. Every project in this series has the same shape - include the library, create the modules, start them, then use them in `loop()`.

## BEFORE YOU START

A. Install the library: in the Arduino IDE choose **Sketch → Include Library → Manage Libraries...**, search for `Modulino` and install **Arduino_Modulino**; also search for `LiquidCrystal I2C` and install it (for the optional LCD). If asked to install its dependencies, choose **Install all**.

B. Choose the board: **Tools → Board → Arduino UNO R4 WiFi**, and select its USB port.

C. Plug the Modulino Buttons and Modulino Pixels into the Uno R4's **Qwiic** connector, chained one after the other. That is all the sketch needs.

D. **Optional:** connect the I2C 1602 LCD's SDA, SCL, 5V and GND to the Arduino's header pins (not Qwiic) - see project 2. If it's there, the sketch prints a message on it; if not, it carries on without.

## THE SKETCH, LINE BY LINE

1
```
#include <Arduino_Modulino.h>
```
Pulls in the Modulino library. This one line gives you a class for every module (`ModulinoButtons`, `ModulinoKnob`, ...) and the `Modulino` object that starts the bus.

2
```
ModulinoButtons buttons;
ModulinoPixels pixels;
```
**One object per module you use.** Name them anything you like. There is only one of each type in the kit, so the library finds each module on the Qwiic bus by itself - no addresses needed.

3
```
Serial.begin(115200);
```
Lets the sketch print messages. Open **Tools → Serial Monitor** (set to 115200) to read them.

4
```
Modulino.begin();
```
Starts the Qwiic bus. **Always call this first**, before any module's `begin()`.

5
```
if (!buttons.begin())
  Serial.println("Buttons not found");
```
Each module has its own `begin()`. It returns `false` if the module isn't found, which is a quick way to spot a loose Qwiic cable.

6
```
Wire.begin();
i2cDevicePresent(0x27)
i2cDevicePresent(0x3F)
```
The LCD is **optional**, so the sketch looks for it first. It sits on the header pins (the `Wire` bus), not Qwiic. `i2cDevicePresent()` just asks "does anything answer at this address?" - 1602 LCDs are at `0x27` or `0x3F`.

7
```
lcd = new LiquidCrystal_I2C(
  lcdAddress, 16, 2);
lcd->init();
lcd->backlight();
```
Only if one answered: create the LCD object (address, 16 columns, 2 rows), start it and switch the backlight on. If the screen stays blank, the contrast dial on the back may need turning.

| | | |
|---|---|---|
| 8 | `lcd->setCursor(0, 0);`<br>`lcd->print("Starter sketch");` | `setCursor(column, row)` picks where the next text goes (both count from 0), then `print()` writes it - just like `Serial.print()`. |
| 9 | `if (buttons.update()) {` | `update()` asks the module for its latest state and returns `true` **only when something changed**. Call it once every time through `loop()`. |
| 10 | `if (buttons.isPressed('A')) {` | Which button? `'A'`, `'B'` or `'C'`. Other modules have their own calls - see the table on the next page. |
| 11 | `pixels.set(0, GREEN, 40);`<br>`pixels.show();` | `set(pixel, color, brightness)`: pixel 0-7, a color name (or `ModulinoColor(r, g, b)`), brightness 0-100. **Nothing lights up until you call** `show()`. **To turn a pixel off, use** `pixels.clear(0)` and call `show()` again. |

## THE SEVEN MODULES AT A GLANCE

| Module | Class name | Common calls |
|---|---|---|
| Buttons | `ModulinoButtons` | `update()`, `isPressed('A')`, `setLeds(a, b, c)` |
| Knob | `ModulinoKnob` | `getDirection()` (+1 / -1 / 0), `isPressed()`, `get()` |
| Buzzer | `ModulinoBuzzer` | `tone(frequency, ms)`, `noTone()` |
| Pixels | `ModulinoPixels` | `set(i, color, brightness)`, `show()`, `clear(i)` |
| Distance | `ModulinoDistance` | `available()`, `get()` (millimeters) |
| Movement | `ModulinoMovement` | `update()`, `getRoll()` / `getPitch()` / `getYaw()`, `getX()` / `getY()` / `getZ()` |
| Thermo | `ModulinoThermo` | `getTemperature()`, `getHumidity()` |

## I2C ADDRESSES

Match what an I2C scanner prints to a module. The Modulino library finds its modules by itself, so the sketch doesn't need these. The 1602 LCD isn't a Modulino; its address depends on the backpack fitted (`0x27` or `0x3F`).

| Address | Name |
|---|---|
| `0x1E` | Modulino Buzzer |
| `0x27 / 0x3F` | 1602 LCD |
| `0x29` | Modulino Distance |
| `0x36` | Modulino Pixels |
| `0x3A` | Modulino Knob |
| `0x3E` | Modulino Buttons |
| `0x44` | Modulino Thermo (HS3003) |
| `0x45` | Modulino Thermo (SHT31) |
| `0x6A` | Modulino Movement |

## THINGS THAT TRIP PEOPLE UP

- **Nothing happens at all?** Check the Qwiic cables are pushed fully in at both ends. A loose connector looks exactly like a missing module, and `begin()` returns `false`.
- **A change is only reported once.** Call `update()` once per loop and share the result - calling it again straight away always says "nothing changed".
- **Avoid long delays.** A long `delay()` freezes the sketch, so it misses button presses and sensor readings. Use `millis()` timing instead (project 2 shows how).
- **The Knob's raw position is arbitrary.** `get()` starts at a random-looking number and counts very finely. Use `getDirection()` instead.
- **No reading when nothing is in range.** The Distance sensor has nothing to report then, so check `available()` before calling `get()`.

## WHERE NEXT

Work through projects 1-6 in order; each one adds a new module or idea. To clear the board between projects, flash `00-hello-world`.

## THE COMPLETE SKETCH

Also in `starter-sketch/starter-sketch.ino` - this page is built from that file.

```
/*
 * Plug and Make Kit - Starter sketch: using the Modulino library
 *
 * The smallest useful Modulino sketch. Press button A on the Modulino
 * Buttons module to light the first pixel on the Modulino Pixels module
 * green; press button B to turn it off again.
 *
 * If an I2C 1602 LCD is connected, the sketch also shows a message on it
 * ("Starter sketch" / "A=on  B=off"). The LCD is optional - without it the
 * sketch works exactly the same.
 *
 * Wiring: plug the Modulino Buttons and Modulino Pixels into the Uno R4's
 * Qwiic connector (chain them together). For the optional LCD, connect its
 * SDA, SCL, 5V and GND to the Arduino's header pins (not the Qwiic connector).
 *
 * Requires the "Arduino_Modulino" and "LiquidCrystal_I2C" libraries
 * (Library Manager).
 */

#include <Arduino_Modulino.h>
#include <LiquidCrystal_I2C.h>

// One object per module. There is only one of each type in the kit, so the
// library finds each module on the Qwiic bus by itself - no addresses needed.
ModulinoButtons buttons;
ModulinoPixels pixels;

// The LCD is optional, so it is only created if we find one (see setup()).
// Its backpack is at address 0x27 or 0x3F depending on the make.
LiquidCrystal_I2C* lcd = nullptr;

// Returns true if something answers on the header I2C bus (Wire) at this address.
bool i2cDevicePresent(byte address) {
  Wire.beginTransmission(address);
  return Wire.endTransmission() == 0;
}

void setup() {
  Serial.begin(115200);

  Modulino.begin();  // start the Qwiic bus - always do this first

  // Each module's begin() returns false if that module isn't found.
  if (!buttons.begin()) Serial.println("Buttons module not found - check the Qwiic cable");
  if (!pixels.begin())  Serial.println("Pixels module not found - check the Qwiic cable");

  // The LCD is on the header pins (Wire), separate from the Qwiic bus.
  Wire.begin();
  byte lcdAddress = 0;
  if (i2cDevicePresent(0x27)) lcdAddress = 0x27;
  else if (i2cDevicePresent(0x3F)) lcdAddress = 0x3F;

  if (lcdAddress != 0) {
    lcd = new LiquidCrystal_I2C(lcdAddress, 16, 2);  // address, columns, rows
    lcd->init();
    lcd->backlight();
    lcd->setCursor(0, 0);  // column, row (both start at 0)
    lcd->print("Starter sketch");
    lcd->setCursor(0, 1);
    lcd->print("A=on  B=off");
  } else {
    Serial.println("LCD not found - skipping (optional)");
  }
}

void loop() {
  // update() returns true only when a button has just been pressed or released.
  if (buttons.update()) {
    if (buttons.isPressed('A')) {
      pixels.set(0, GREEN, 40);  // pixel number, color, brightness (0-100)
      pixels.show();             // nothing changes until you call show()
    } else if (buttons.isPressed('B')) {
      pixels.clear(0);           // turn pixel 0 off
      pixels.show();
    }
  }
}
```